Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

GitHub Repository

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

2.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel
The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)
The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)
The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel
The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel
This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel
The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel
Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel
The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel
This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel
This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

3. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

4. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$\begin{aligned} & (3 \times 3 \times 3 + 1) \times 16 \\ = & (27 + 1) \times 16 \\ = & 448 \end{aligned}$$

Therefore,
Total trainable parameters = 448.

6. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

7. Comparison between Max Pooling and Average Pooling

Comparison Table:

Metric	**Max Pooling**	**Average Pooling**
Test Loss	0.9378	0.9537
Test Accuracy	0.6715	0.6684
1st Pooling Output Size	16×16	16×16
2nd Pooling Output Size	8×8	8×8

8. Mandatory Plots

8.1 Sample Images

Figure 1: Sample images from the CIFAR-10 dataset.

Interpretation: The images show different classes from the CIFAR-10 dataset, such as frog, truck, deer, automobile, bird, horse, ship, and cat. The images are small and have low resolution, but the objects are still recognizable based on their visual features.

8.2 Class Distribution

Figure 2: Distribution of classes in the training set.

Interpretation: The class distribution is almost equal for all 10 CIFAR-10 classes, with around 5,000 images in each class. This shows that the dataset is well-balanced and does not have any major class imbalance. .

8.3 Training Accuracy

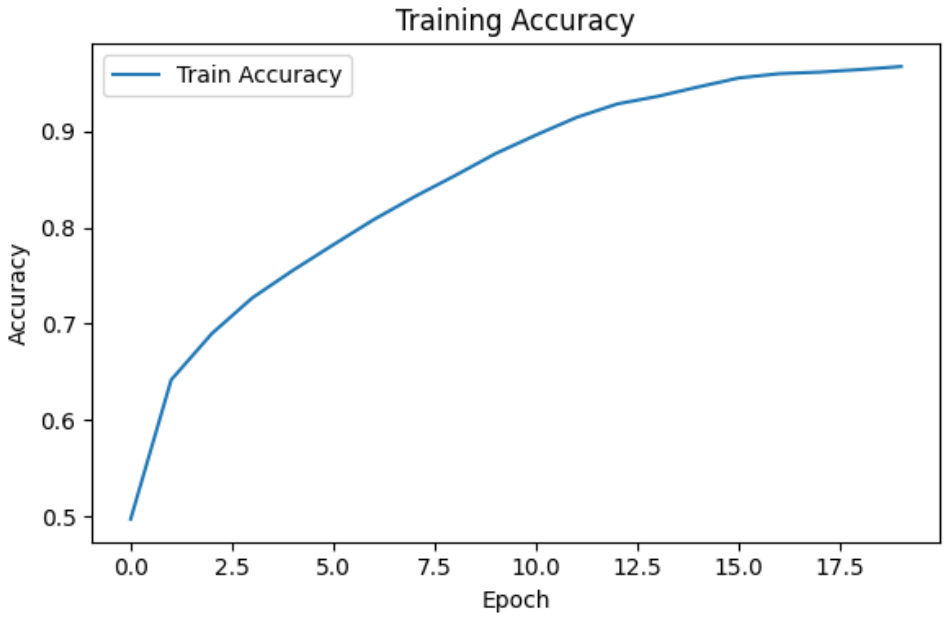

Figure 3: Training accuracy over epochs.

Interpretation: The training accuracy increases steadily ,as the number of epochs increases. This shows that the model is learning well and improving its performance over time.

8.4 Validation Accuracy

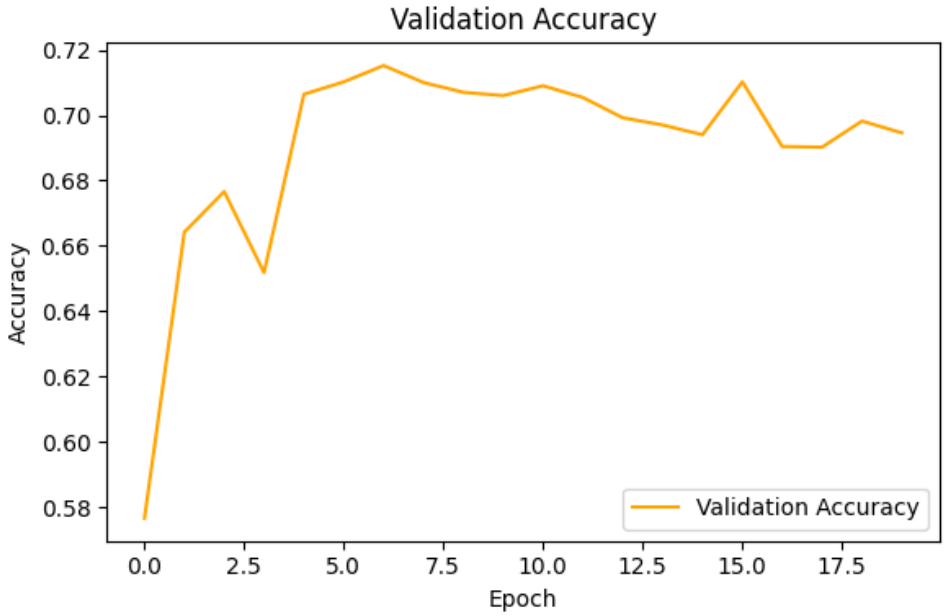

Figure 4: Validation accuracy over epochs.

Interpretation: The validation accuracy increases in the initial epochs and reaches its highest point (71.5%) around the 7th epoch. After that, it fluctuates and gradually declines, showing

that the model's performance on unseen data has started to plateau while training accuracy keeps climbing.

8.5 Training Loss

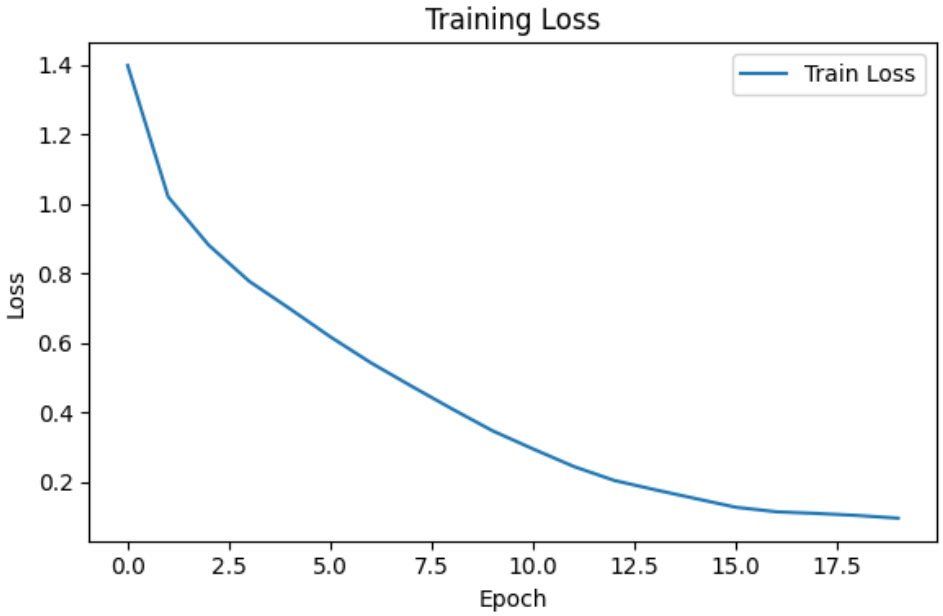

Figure 5: Training loss over epochs.

Inference: The training loss decreases steadily as the number of epochs increases. This shows that the model is learning effectively and making fewer errors during training.

8.6 Validation Loss

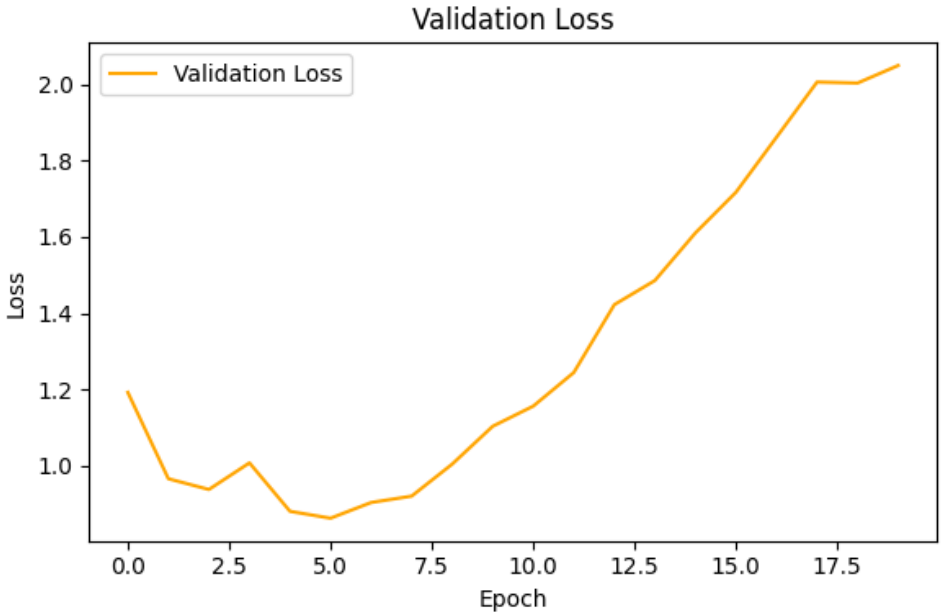

Figure 6: Validation loss over epochs.

Interpretation: The validation loss decreases initially and reaches its lowest point (0.862) around the 6th epoch. After that, it rises steadily even as training loss keeps falling, which is a clear sign that the model is overfitting the training data.

8.7 Feature Maps

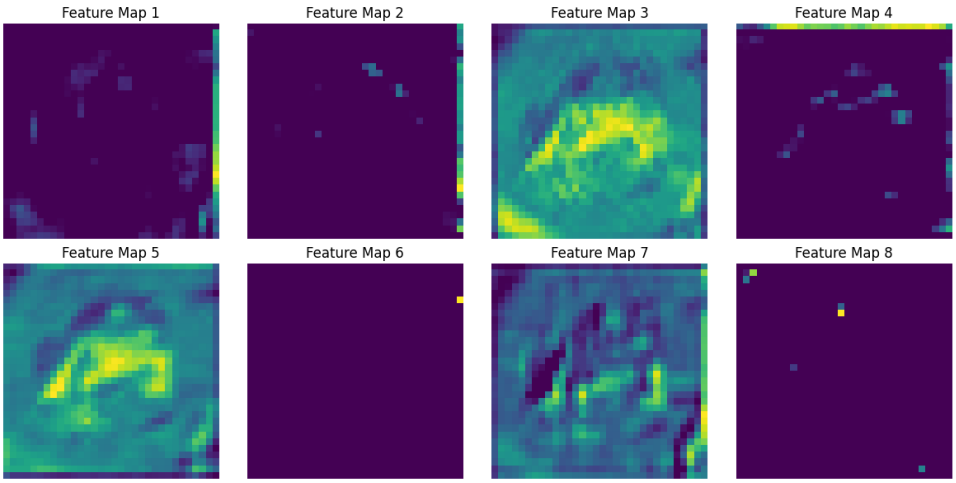

Figure 7: Feature maps generated after the first convolution layer.

Interpretation: Each feature map highlights a different aspect of the input image, such as edges, contours, or textured regions. Some feature maps (e.g., Map 6) are almost entirely inactive, showing that not every filter learns a useful feature at this stage.

8.8 Confusion Matrix

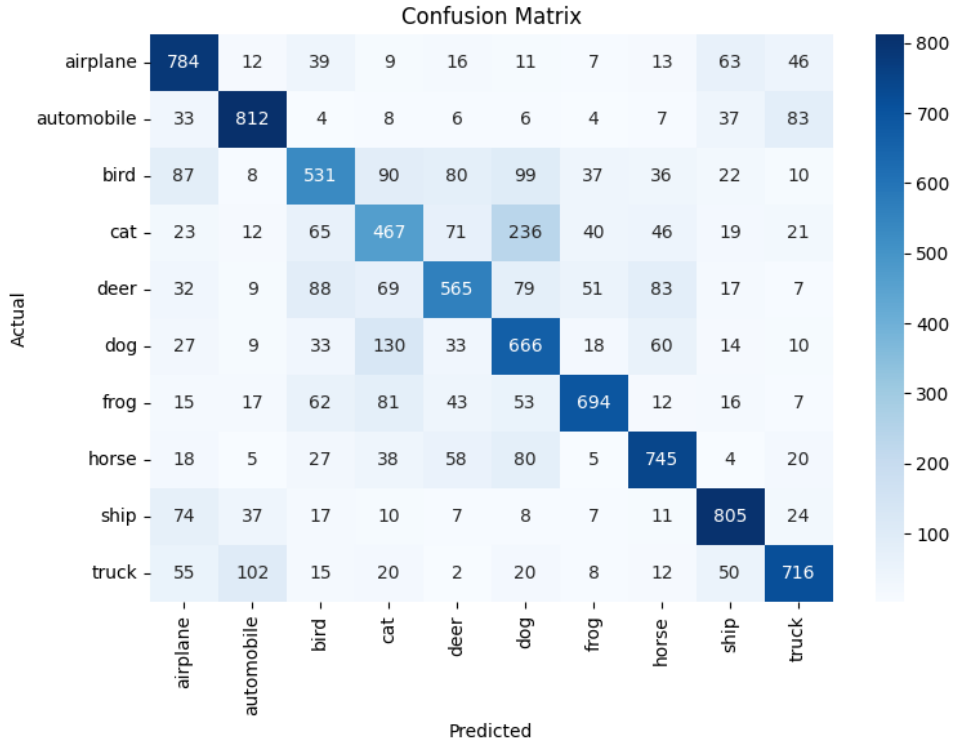

Figure 8: Confusion matrix of the evaluated model.

Interpretation: The confusion matrix shows that the model performs well for classes like Automobile, Frog, Ship, and Horse. However, some confusion can be seen between similar classes, especially Cat–Dog and Truck–Automobile.

9. Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**
   Output Size:

   The output size is calculated using the convolution formula and is **32 × 32**. This reduces the spatial dimensions while preserving important features.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**
   The convolution layer has **1,792 trainable parameters**, including the weights and biases for all 64 filters.

3. **Compare ReLU and Sigmoid activation functions.**
   ReLU is faster and commonly used in CNN hidden layers, while Sigmoid is mainly used for binary classification. ReLU also helps reduce the vanishing gradient problem.

4. **Replace Max Pooling with Average Pooling and compare the results.**
   In this experiment, Max Pooling gave a slightly higher test accuracy (67.15%) than Average Pooling (66.84%), with a lower test loss as well. Both methods perform comparably overall, but Max Pooling has a small edge here since it retains the strongest activations rather than smoothing them out.

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**

   Increasing filters from 16 to 64 helps the model learn more complex features and can improve accuracy. However, it also increases computation time and memory usage.

10. Results

The evaluation metrics for the primary Convolutional Neural Network (Max Pooling model) are recorded below:

- **Training Accuracy:** 96.68%
- **Testing Accuracy: 67.85%**
- **Precision (macro):** 0.6808
- **Recall (macro):** 0.6785
- **F1-score (macro):** 0.6775
- **Number of Parameters:** 545,098

11. Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?**
   Convolution preserves the spatial structure of images and detects local features like edges and textures. It also uses fewer parameters than fully connected layers.

2. **How does stride affect the feature map size?**
   A larger stride moves the filter by more pixels, resulting in a smaller feature map. A smaller stride produces a larger feature map

3. **What is the role of padding?**
   Padding adds extra pixels around the image to control the output size. It also helps preserve information from the edges of the image.

4. **Why is pooling used?**
   Pooling reduces the spatial dimensions of feature maps and decreases computation. It also helps the model focus on the most important features

5. **How do feature maps represent image characteristics?**
   Feature maps show the features detected by convolution filters, such as edges, shapes, textures, and patterns. Different filters detect different characteristics of the image.

6. **Why do CNNs require fewer parameters than MLPs?**
   CNNs use shared weights and local connections, so the same filter is applied across the image. This greatly reduces the number of trainable parameters compared to MLPs.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.